PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

Railway Reservation System (RRS)

Version 1.0 approved

Course: UE24CS341A - Software Engineering | Group Number: Team 5

Technology Stack: C / C++ (TUI via ncurses, optional decoupled MERN/API wrapper)

Prepared by: Ridhima Jain (PES1UG24CS373) · S S Adhithya Sriram (PES1UG24CS393) · Saathvik Shenoy Padubidri (PES1UG24CS394) · Samhitha S (PES1UG24CS411)

Table of Contents

Table of Contents ii
Revision History 4
1. Introduction 5
1.1 Purpose 5
1.2 Intended Audience and Reading Suggestions 5
1.3 Product Scope 5
1.4 References 5
2. Overall Description 6
2.1 Product Perspective 6
2.2 Product Functions 6
2.3 Scope 6
2.4 User Classes and Characteristics 6
2.5 Operating Environment 7
2.6 Design and Implementation Constraints 7
2.7 Assumptions and Dependencies 7
3. External Interface Requirements 8
3.1 User Interfaces 8
3.2 Software Interfaces 8
3.3 Communications Interfaces 8
4. Analysis Models 9
4.1 Use Case Summary 9
5. System Features 10
5.1 User Access & Profile Module 10
5.2 Inventory & Schedule Module (Admin) 11
5.3 Search Engine, Fare & Eco-Optimization Module 11
5.4 Booking & Queue Management Module 12
6. Other Nonfunctional Requirements 14
6.1 Performance and Other Measurable NFRs 14
6.2 Safety Requirements 14
6.3 Security Requirements 14
6.4 Software Quality Attributes 15
6.5 Business Rules 15
7. Validation Specification – Initial Planning 16
8. Project Plan 17

8.1 Work Breakdown Structure..... 17
8.2 Sprint Schedule..... 17
9. Tool Stack..... 18
10. Other Requirements..... 19
Appendix A: Glossary..... 20
Appendix B: Field Layouts..... 21
Appendix C: Requirement Traceability Matrix..... 23

Revision History

Name	Date	Reason For Changes	Version
Team 5	06.09.2026	Initial SRS draft created	1.0
Team 5	28.09.2026	Added security objectives, NFR-009, corrections	1.1

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of the Railway Reservation System, a C/C++ backend application that automates train scheduling, ticket booking, waitlist/queue management and administrative operations for a rail network. This SRS covers the full scope of the system as proposed in the approved project synopsis: passenger-facing booking and search functionality, administrator-facing inventory and schedule management, and the supporting non-functional and security requirements.

1.2 Intended Audience and Reading Suggestions

This document is intended for the four members of Team 5 (developers/testers), the course instructor and teaching assistants (evaluators), and any future maintainer of the codebase. Developers should read Sections 3- 6 closely before implementing their owned module. Evaluators should read Sections 2 and 5 to understand functional scope, and Appendix C to trace requirements to design, code and tests. The document is organized following the standard SRS template: overall description, external interfaces, analysis models, system features grouped by module ownership, and non-functional requirements.

1.3 Product Scope

The Railway Reservation System digitizes the end-to-end passenger journey — from searching for a route, comparing fare/time/eco-efficiency trade-offs, booking a ticket in a preferred class, and, if necessary, being placed on and automatically promoted from a waitlist. It also gives railway administrators a controlled way to define trains, routes and station halts, and to monitor bookings and revenue. The core business logic (route graph, fare engine, booking/waitlist state machine, RBAC) is implemented in C/C++; a Text User Interface (TUI) built with ncurses, or an optionally decoupled thin API wrapper, presents this logic to end users without altering the underlying engine. The project supports the course objective of demonstrating the full software development lifecycle using Agile methodology.

1.4 References

- PES University, Jackfruit - Software Engineering Project Guidelines 2026 (UE24CS341A).

- PES University, Software Engineering - Level 3 Mini-Project Student Submission Guide & Exemplar.
- Team 5, Railway Reservation System - Synopsis / Project Proposal, v1.0.
- ncurses Programming HOWTO (for TUI layer reference).

2. Overall Description

2.1 Product Perspective

RRS is a new, self-contained academic project; it is not a follow-on to an existing deployed system, though it models the domain of real-world railway reservation platforms (e.g. IRCTC-style booking). The system is a standalone application with a layered architecture: a Text User Interface (or thin API wrapper) at the front, a C/C++ core engine implementing route search, fare computation, booking and queue logic in the middle, and a file/lightweight-database persistence layer at the back. External systems interfaced with are limited to a mocked payment confirmation step; no live payment gateway is integrated in this academic scope.

2.2 Product Functions

- Passenger registration, secure login and role-based session management.
- Graph-based route search across direct and multi-leg (connected) journeys.
- Filtering and sorting of train options by fare, travel time and eco-routing metrics.
- Ticket booking with class selection and unique PNR generation.
- Automatic waitlist queueing when a class/train is at capacity.
- Ticket cancellation with automatic waitlist promotion to free seats.
- Administrative management of trains, routes, station halts and seat inventory.
- Administrative operational reporting (bookings, revenue, occupancy).

2.3 Scope

In Scope	Out of Scope
Passenger registration and login	Real-money payment settlement / bank integration
Graph-based direct and multi-leg route search	Live GPS train tracking
Dynamic fare and eco-score calculation	Multi-currency / international rail networks
Ticket booking, PNR generation, class selection	Physical ticket printing / kiosk hardware integration

Waitlist queueing and automatic promotion	Loyalty/rewards or subscription programs
Ticket cancellation and refund status (mocked)	Real-time crowd/occupancy sensing on trains
Admin management of trains, routes, halts, capacities	Multi-tenant deployment across multiple railway operators
Admin operational reporting (bookings, revenue)	Mobile native apps (only TUI and optional web wrapper are in scope)

2.4 User Classes and Characteristics

User Class	Privileges	Characteristics
Passenger	Register, log in, search routes, book/cancel tickets, view own PNR status	Non-technical, general public; primary and most frequent user class; expects a simple, fast TUI flow
Railway Administrator	Manage trains, routes, halts, seat inventory; view operational reports	Smaller, trusted user base; higher privilege; assumed to have basic system familiarity
Guest (unauthenticated)	View public schedule information only	Least privileged; must register/login before booking

2.5 Operating Environment

The RRS backend is developed in C/C++ and targets a Linux development/evaluation environment. The TUI layer uses the ncurses library for terminal rendering. If the team implements the optional decoupled wrapper, a lightweight REST API (e.g. served over HTTP) exposes the same C/C++ core to a MERN-based front end, communicating over JSON. Persistence uses either flat files or an embedded/lightweight database, consistent across both interface options so the core engine is not tied to either presentation.

2.6 Design and Implementation Constraints

- Core business logic must be implemented in C/C++, independent of the presentation layer (TUI vs API wrapper).
- Route search must use graph-based data structures; schedule/route storage must use tree-based structures (e.g. B-trees) as proposed in the synopsis.
- Agile methodology must be followed throughout, with Git/GitHub as the version-control and collaboration tool.
- Each module owner's code must remain loosely coupled from other modules through well-defined internal APIs/ABIs so modules can be developed, tested and reviewed independently.
- No real payment gateway integration is permitted in this academic scope; payment must be mocked.
- The system must be demonstrable via a working TUI within the constraints of a terminal environment.

2.7 Assumptions and Dependencies

- It is assumed that train, station and route master data will be seeded by the team for demonstration purposes rather than sourced from a live external railway database.
- It is assumed ncurses (or an equivalent terminal UI library) is available on the target evaluation machine; if unavailable, the fallback is the decoupled API + web wrapper.
- The eco-routing/carbon-footprint metric depends on approximate, published per-passenger-km emission factors for rail travel rather than live telemetry, since no such live data source exists for this project.
- The project depends on GitHub Actions (or an equivalent CI tool) being available for the CI/CD pipeline required by the course guidelines.

3. External Interface Requirements

3.1 User Interfaces

The primary interface is a menu-driven Text User Interface (TUI) built with ncurses, offering distinct menu flows for Passenger and Administrator roles. Standard conventions apply across all screens: a persistent header showing the logged-in user and role, numbered/arrow-key menu navigation, a consistent 'back'/'cancel' key, and inline validation messages shown immediately below the offending field. If the optional MERN/API wrapper is implemented, an equivalent web UI will mirror the same screen flows and validation rules, calling the same underlying C/C++ engine through its API so behaviour stays consistent across both interfaces.

3.2 Software Interfaces

Component	Interfaces With	Purpose
Core Engine (C/C++)	Persistence layer (files / embedded DB)	Read/write train, route, user, booking and waitlist records
Core Engine (C/C++)	TUI (ncurses) or REST API wrapper	Expose search, booking, cancellation and admin functions via an internal function-call API or HTTP endpoints
Booking Module	Mock Payment Service	Simulate authorization and return success/failure to gate PNR confirmation
CI/CD pipeline	GitHub Actions, testing tools (gtest/Unity/Check, cppcheck)	Automated build, unit/integration test execution and static analysis on each push/PR

3.3 Communications Interfaces

For the TUI-only deployment, all communication is local process I/O between the ncurses front end and the C/C++ core - no network protocol is required. If the optional decoupled wrapper is built, the MERN front end communicates with the API layer over HTTP using JSON request/response payloads, and the API layer invokes the C/C++ core via bindings or inter-process calls. All administrator actions and failed-login attempts are logged locally for auditability.

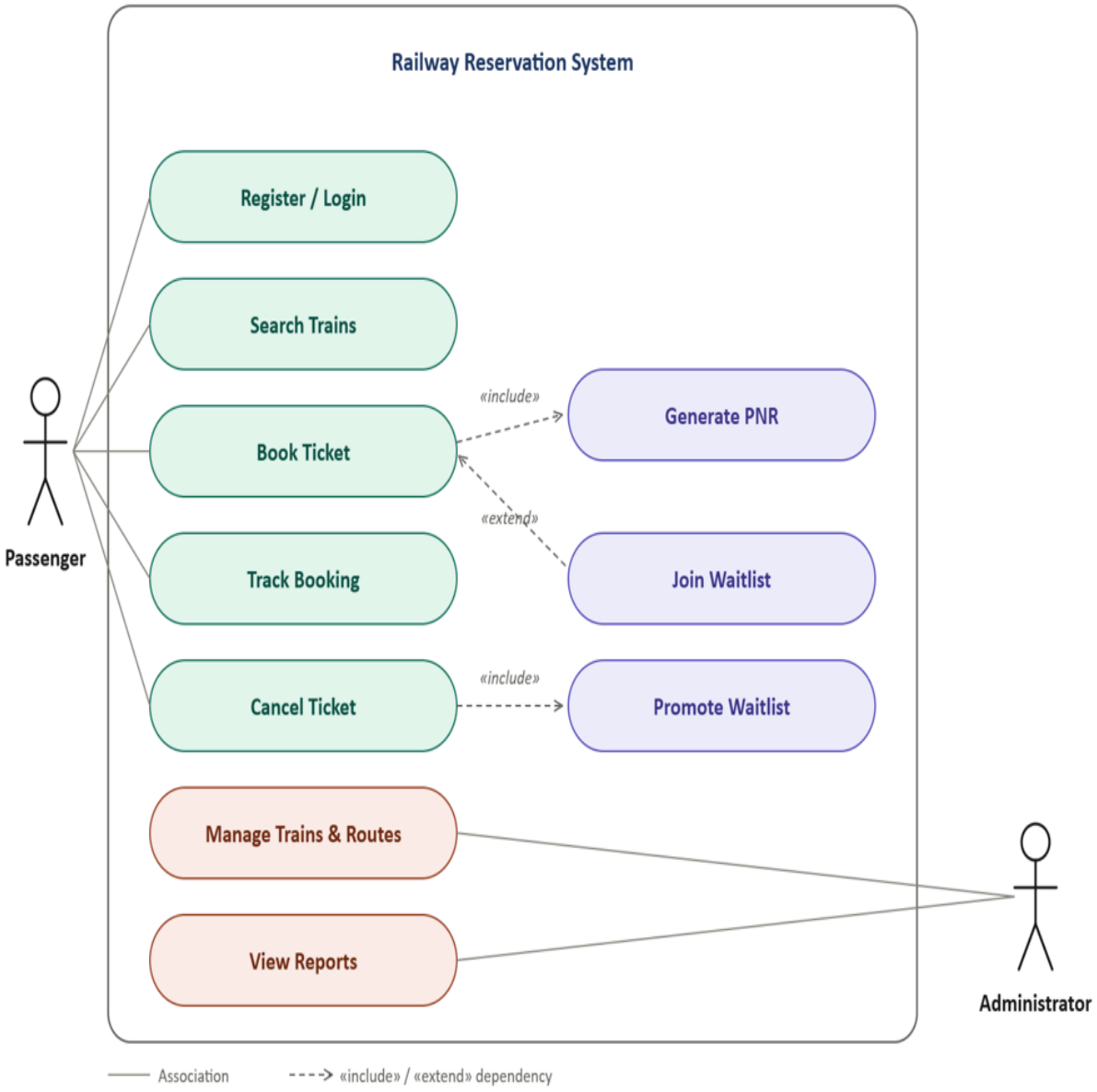
4. Analysis Models

This section will contain the Use Case Diagram and the domain model (Entity/Class diagram) for RRS. A textual use-case summary is provided below as the initial analysis model; the diagrammatic UML (use case, class, sequence for Book Ticket and Cancel/Waitlist-Promote, activity for Book Ticket, and component diagram) will be added in Deliverable 3 - Design, per the project flow.

4.1 Use Case Summary

UC ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Passenger	Application available	Authenticated session with role assigned
UC-02	Search Trains	Passenger	Logged in	Ranked list of direct/connected routes with fare, time, eco-score
UC-03	Book Ticket	Passenger	Logged in; valid search result selected	PNR generated; seat confirmed or waitlisted
UC-04	Track / View Booking	Passenger	Valid PNR exists	Current booking/waitlist status displayed
UC-05	Cancel Ticket	Passenger	Confirmed or waitlisted PNR exists	Ticket cancelled; seat released; waitlist auto-promoted
UC-06	Manage Trains & Routes	Administrator	Admin authenticated	Train/route/halt/capacity data added or updated
UC-07	View Operational Reports	Administrator	Admin authenticated	Booking and revenue report displayed

4.2 UML Diagram



5. System Features

Functional requirements are organized by system feature, aligned one-to-one with the four owned functional modules from the project synopsis so that ownership, design, implementation and testing stay traceable to a single accountable team member. UI, database interaction and unit testing are treated as secondary responsibilities under each module, not as separate functional features, per the course guidelines.

5.1 User Access & Profile Module

Owner: Ridhima Jain (PES1UG24CS373) - Secondary: user profile UI flows, session management

5.1.1 Description and Priority

Handles onboarding for both passengers and administrators: registration, secure credential verification, and Role-Based Access Control (RBAC) that gates every other module's endpoints by role.

Priority: High

5.1.2 Stimulus/Response Sequences

- A new user submits registration details → system validates uniqueness and stores a hashed credential, returning a confirmation.
- A registered user submits login credentials → system authenticates and returns a session bound to the user's role (Passenger/Administrator).
- A Passenger session attempts to call an Administrator-only function → system rejects the call with an authorization error.

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a new passenger to register using a unique email/username and a password meeting the defined complexity rule.	High	Functional test
FR-002	The system shall reject registration attempts using an email/username already on record.	High	Negative test
FR-003	The system shall authenticate a user's credentials and establish a session tagged with the user's role before granting access to any booking or admin function.	High	Functional test
FR-004	The system shall enforce Role-Based Access Control so that only Administrator-role sessions can invoke train, route or reporting management functions.	High	Authorization test

FR-005	The system shall allow an authenticated passenger to view and update their own profile details.	Medium	Functional test
FR-006	The system shall terminate a session after a defined period of inactivity or on explicit logout.	Medium	Functional test

5.2 Inventory & Schedule Module (Admin)

Owner: S S Adhithya Sriram (PES1UG24CS393) - Secondary: admin dashboard UI, database mock schema generation

5.2.1 Description and Priority

Provides the administrator backend for defining and maintaining the operational data the rest of the system depends on: train numbers, multi-station routes, departure/arrival times, station halts, and per-class seat capacities.

Priority: High

5.2.2 Stimulus/Response Sequences

- An authenticated administrator submits a new train definition with route and capacities → system validates and creates the train/route record.
- An administrator edits an existing route's station halts → system validates the change does not break in-progress bookings and updates the schedule.
- An administrator disables a train → the system prevents new bookings against it while preserving existing PNR history.

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-007	The system shall allow an administrator to create a new train with a unique train number, an ordered list of station halts, and per-class seat capacities.	High	Functional test
FR-008	The system shall allow an administrator to modify an existing train's route, halts, timings or capacities.	High	Functional test
FR-009	The system shall allow an administrator to disable (retire) a train so that it can no longer be booked while historical bookings remain viewable.	Medium	Functional test

FR-010	The system shall validate that a train's ordered station halts form a connected path usable by the route-search graph before saving.	High	Functional + integration test
FR-011	The system shall reject a capacity update that would reduce a class's capacity below the number of currently confirmed bookings in that class.	High	Negative test

5.3 Search Engine, Fare & Eco-Optimization Module

Owner: Samhitha S (PES1UG24CS411) - Secondary: core algorithm unit testing, performance benchmarking

5.3.1 Description and Priority

Implements the graph-based route search across the train network (direct and multi-leg/connected journeys) and computes dynamic fares and eco-routing metrics (fuel efficiency and carbon footprint per passenger) used to filter and rank results for passengers.

Priority: High

5.3.2 Stimulus/Response Sequences

- A passenger submits an origin, destination and travel date → system returns ranked direct and connected journey options within a bounded response time.
- A passenger applies a fare, duration or eco-score filter → system re-ranks the existing result set without a full re-search.
- The engine computes a fare for a candidate route → result reflects distance, travel class, and applicable passenger concessions.

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-012	The system shall search the train network graph and return all direct routes between a given origin and destination.	High	Functional test
FR-013	The system shall return connected (multi-leg) journey options when no direct route exists or when a connected option is faster/cheaper.	High	Functional test
FR-014	The system shall calculate a dynamic fare for each route option based on distance, travel class and applicable passenger concessions.	High	Unit + functional test

FR-015	The system shall calculate an eco-routing score (estimated fuel efficiency and carbon footprint per passenger) for each route option.	Medium	Unit test
FR-016	The system shall allow the passenger to sort and filter search results by fare, total travel time, or eco-score.	High	Functional test
FR-017	The system shall return search results for a bounded network size within the response-time limit defined in NFR-001.	High	Performance test

5.4 Booking & Queue Management Module

Owner: Saathvik Shenoy Padubidri (PES1UG24CS394) - Secondary: payment state mock integration, waitlist UI views

5.4.1 Description and Priority

Implements the transactional booking logic: decrementing available seats, generating unique PNRs, routing overflow demand into a structured waitlist queue, and processing cancellations by automatically promoting the next waitlisted passenger into any freed seat.

Priority: High

5.4.2 Stimulus/Response Sequences

- A passenger confirms a selected route and class with successful mock payment → system creates a booking, assigns a unique PNR, and decrements seat availability.
- A passenger attempts to book a class that is at capacity → system enqueues the request onto that train/class's waitlist and returns a waitlist position instead of a confirmed PNR.
- A passenger cancels a confirmed ticket → system releases the seat and automatically promotes the head of the corresponding waitlist to CONFIRMED.

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-018	The system shall create a booking and generate a unique PNR only after mock payment authorization succeeds.	High	Integration test
FR-019	The system shall not create a confirmed booking when mock payment authorization fails.	High	Negative + integration test
FR-020	The system shall automatically place a booking request onto the class's waitlist queue when no	High	Functional test

	seat is available, preserving first-come-first-served order.		
FR-021	The system shall allow a passenger to cancel a confirmed or waitlisted ticket.	High	Functional test
FR-022	The system shall automatically promote the head of the waitlist to CONFIRMED and issue a PNR when a seat is freed by a cancellation.	High	Integration test
FR-023	The system shall display booking/waitlist status as one of: WAITLISTED, CONFIRMED, or CANCELLED.	High	System test
FR-024	The system shall prevent a duplicate confirmed booking from being created if a passenger retries after a payment timeout.	High	Negative/integration test

6. Other Nonfunctional Requirements

6.1 Performance and Other Measurable Non-Functional Requirements

The table below consolidates the measurable non-functional requirements across performance, availability, usability, reliability, maintainability and testability, each stated with a verifiable target rather than a vague qualitative goal.

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of route-search requests shall return results within 2 seconds for a network of up to 200 stations and 500 trains under the defined test load.	Performance test
NFR-002	Performance	Booking confirmation (post mock-payment) shall complete within 1 second under normal load.	Performance test
NFR-003	Availability	The application shall be available at least 99% of the defined evaluation/operating window, excluding planned maintenance.	Operational review
NFR-004	Usability	A new passenger shall be able to complete search-to-booking in no more than 6 primary TUI screens.	Usability walkthrough

NFR-005	Reliability	A booking shall not be created twice when a passenger retries after a mock-payment timeout.	Negative/integration test
NFR-006	Maintainability	Business logic (search, fare/eco, booking/waitlist, RBAC) shall be separated into independently identifiable C/C++ modules with automated unit tests.	Code review
NFR-007	Testability	Core engine functions shall expose testable interfaces callable without the ncurses TUI or a live payment service.	Unit/integration test
NFR-008	Scalability	The route graph and waitlist queue structures shall support at least 500 trains and 10,000 concurrent bookings without redesign.	Load test
NFR-009	Integrity	No seat shall be allocated to more than one confirmed booking; confirmed bookings per train/class shall never exceed its capacity, including under 50 simulated concurrent booking requests.	Concurrency/integration test

6.2 Safety Requirements

As an academic simulation with no live payment or physical train control, there are no life-safety hazards. The primary 'safety' concern is data integrity: the system shall not lose or corrupt a confirmed booking record, and shall not allow the same seat to be double-booked (see NFR-009) .

6.3 Security Requirements

Security Objectives

- **SO-1:** Protect the confidentiality of user credentials and personal data.
- **SO-2:** Ensure only authorised roles can perform privileged actions.
- **SO-3:** Preserve booking integrity (no forged, duplicate or double-booked tickets).
- **SO-4:** Provide auditability of administrator actions and failed logins.

ID	Security Requirement	Validation Approach	Security Objective
SEC-001	Passwords shall never be stored in plaintext; a salted hash shall be used.	Code/configuration review	SO-1

SEC-002	Role-based access shall prevent Passenger-role sessions from invoking Administrator-only functions.	Authorization tests	SO-2
SEC-003	The system shall validate and sanitize all user-supplied fields (registration, search, booking) before processing.	Negative/security tests	SO-3
SEC-004	Expired or invalid session tokens shall be rejected on every protected function call.	Security test	SO-2
SEC-005	Mock payment details shall not be persisted beyond what is required to record success/failure against a booking.	Design review	SO-1
SEC-006	Failed login attempts and unauthorized access attempts shall be logged for audit.	System test	SO-4

6.4 Software Quality Attributes

Attribute	Target	Rationale
Maintainability	Business logic (search, fare/eco, booking/waitlist, RBAC) shall be separated into independently testable C/C++ modules with automated unit tests.	Enables 4 owners to work in parallel without merge conflicts
Testability	Core engine functions shall be callable and testable independently of the ncurses TUI.	Allows gtest/pytest suites to run headless in CI
Reliability	An order/booking shall never be created twice for the same successful payment retry.	Prevents double-charging / double-booking under network retries
Usability	A new passenger shall be able to complete a basic search-to-booking flow within 6 TUI screens.	Keeps the terminal interface approachable for non-technical users
Portability	The C/C++ core shall build and run on a standard Linux environment without platform-specific extensions beyond ncurses.	Ensures reproducible evaluation across team machines

6.5 Business Rules

- Only an authenticated passenger may create, view or cancel their own bookings; passengers may not view or cancel another passenger's PNR.
- Only an authenticated administrator may create, edit or disable trains, routes, halts and capacities.
- A booking may not be confirmed unless mock payment authorization has succeeded.
- A cancelled ticket's seat must be offered to the head of the corresponding waitlist before it is made generally available.
- A train's seat capacity for a class may not be reduced below the number of currently confirmed bookings in that class.
- Waitlist promotion follows strict first-come-first-served order per train and class.

7. Validation Specification – Initial Planning

At this requirements stage, validation planning is forward-looking: it explains how each requirement category will eventually be validated once implemented, including planned security/penetration-style testing. This is not the final test/validation report — that is produced in Deliverable 5 once execution results (Actual Result / Pass-Fail) are available.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests against each FR	Book a ticket end-to-end and verify a PNR is issued
Security	Authentication/authorization tests + basic security/negative testing	Passenger session attempts an admin-only endpoint
Performance	Load / response-time testing under the defined test load	Concurrent route-search requests against NFR-001
Usability	Task-based walkthrough with a representative new user	New passenger completes a booking in ≤ 6 screens
Reliability	Failure / retry scenario testing	Payment timeout followed by a retry
Unit	Automated unit tests with coverage reporting per module	Fare calculation, waitlist promotion logic

8. Project Plan

The plan below extends the short-term (4-week) sprint plan from the synopsis into a full work-breakdown structure (WBS) with ownership and estimated effort, so the requirements stage output can be scheduled through to final validation and demo, consistent with the Agile methodology mandated for this course.

8.1 Work Breakdown Structure

Work Package	Tasks	Owner	Estimated Effort (person-days)	Output
WP1	Requirements gathering, use cases, SRS finalization	All	4	SRS v1.0
WP2	Route graph design, RBAC design, DB/field layout	All	3	Architecture & field layout
WP3	User Access & Profile module (registration, login, RBAC)	Ridhima	6	Working feature
WP4	Inventory & Schedule module (trains, routes, halts, capacity)	Adhithya	6	Working feature
WP5	Search Engine, Fare & Eco-Optimization module	Samhitha	7	Working feature
WP6	Booking & Queue Management module	Saathvik	7	Working feature
WP7	Integration, CI/CD, code reviews, static analysis, security validation	All	5	Test/validation evidence
WP8	Final system validation, maintenance plan, report & demo prep	All	4	Final report + demo

8.2 Sprint Schedule

Sprint	Planned Output	Acceptance Evidence

Sprint 1	SRS finalized, Git repo set up, user registration workflow	User stories + Git repo + working login
Sprint 2	Route graph / B-tree data structures, search algorithm, eco-model	Demo + unit tests
Sprint 3	Transactional booking logic, PNR generation, waitlist state machine	Integration tests + PRs
Sprint 4	TUI integration, CI/CD pipeline, unit + system testing, validation report	System demo + test evidence

9. Tool Stack

Activity	**Recommended / Example Tool**	**Evidence to Submit**
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions / Jenkins	Pipeline run log / screenshot
Build system	Make / CMake	Build scripts and reproducible build evidence
Testing	gtest (C++) / Unity or Check (C)	Test results and coverage report
Static analysis	SonarQube, cppcheck, or similar	Analysis report
TUI library	ncurses	Compiled TUI demo
Design	Draw.io / StarUML / PlantUML	UML diagrams

10. Other Requirements

- Database/Data Retention: Booking and cancellation history shall be retained for the duration of the academic evaluation period; no long-term archival/purge policy is required for this scope.

- Legal: No real personal or payment data shall be collected; all seeded passenger and payment data used for demonstration shall be synthetic.
- Localization: The system targets English-language output only for this release.
- Reuse: Standard C/C++ data-structure implementations (graph, B-tree) may be adapted from open-source reference implementations, provided each team member can explain the adapted code as required by the course guidelines on LLM/third-party code use.

Appendix A: Glossary

Term	Definition
PNR	Passenger Name Record - the unique identifier assigned to a confirmed or waitlisted booking.
Waitlist	An ordered queue of booking requests held when a train/class is at full seat capacity, promoted on cancellation.
RBAC	Role-Based Access Control - restricting system functions based on a user's assigned role (Passenger/Administrator).
Multi-leg journey	A route between origin and destination composed of two or more connected train segments.
Eco-score	A computed metric approximating fuel efficiency and carbon footprint per passenger for a given route option.
Halt	A station at which a given train stops along its route.
Class	A travel category on a train (e.g. Sleeper, AC 3-Tier, AC 2-Tier) with its own seat capacity and fare.

Appendix B: Field Layouts

Field layouts and report requirements for the core entities are given below; these will be refined into the physical schema during the Design phase.

B.1 Passenger

Field	Length	Data Type	Description	Mandatory
passenger_id	36	UUID/String	Unique passenger identifier	Y
email	100	String	Unique login email/username	Y
password_hash	64	String	Salted hash of the password	Y
full_name	100	String	Passenger display name	Y
role	20	Enum	PASSENGER or ADMIN	Y

B.2 Train / Route

Field	Length	Data Type	Description	Mandatory
train_id	10	String	Unique train number	Y
route_halts	—	Ordered list	Ordered list of station codes	Y
class_capacity	—	Map<Enum,Int>	Seat capacity per travel class	Y
departure_time	8	Time	Scheduled departure at origin	Y
status	20	Enum	ACTIVE or RETIRED	Y

B.3 Booking

Field	Length	Data Type	Description	Mandatory
pnr	12	String	Unique booking identifier	Y
passenger_id	36	UUID/String	Reference to passenger	Y
train_id	10	String	Reference to train	Y
class	20	Enum	Booked travel class	Y
status	20	Enum	WAITLISTED / CONFIRMED / CANCELLED	Y
fare_amount	10,2	Decimal	Computed fare charged	Y

B.4 Sample Report Requirements

Booking Report	Revenue Report
PNR	Date
Passenger Name	Train ID

Train ID / Route	Total Bookings
Class	Total Cancellations
Status	Net Revenue
Booking Date	Waitlist Conversions

Appendix C: Requirement Traceability Matrix

The matrix below will be completed progressively as Architecture, Design, Code and Test artifacts are produced in Deliverables 3 and 4. Requirement IDs are seeded here from Section 5 so that traceability starts at the requirements stage as required by the course guidelines.

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID
1	FR-001	Passenger registration	TBD	TBD	TBD	TBD
2	FR-004	RBAC enforcement	TBD	TBD	TBD	TBD
3	FR-012	Direct route search	TBD	TBD	TBD	TBD
4	FR-014	Dynamic fare calculation	TBD	TBD	TBD	TBD
5	FR-018	Booking + PNR generation	TBD	TBD	TBD	TBD
6	FR-022	Waitlist auto-promotion	TBD	TBD	TBD	TBD
7	SEC-002	Admin-only authorization	TBD	TBD	TBD	TBD

8	NFR-001	Search response time	TBD	TBD	TBD	TBD

SRS v1.2 Revision (13 November 2026)

This revision completes **Appendix C: Requirement Traceability Matrix**, which version 1.1 left as TBD until the architecture, design, code and test artifacts existed. No requirement has been added, removed or reworded; the requirement text in Sections 5 and 6 is unchanged.

Revision History (continued)

Name	Date	Reason For Changes	Version
Team 5	06.09.2026	Initial SRS draft created	1.0
Team 5	28.09.2026	Added security objectives, NFR-009, corrections	1.1
Team 5	13.11.2026	Appendix C completed with architecture, design, code and test references and results after Sprints 1-3; implementation notes added	1.2

Implementation notes against this SRS

Every requirement is implemented and validated (see Appendix C). Points an evaluator should know:

- **FR-005, FR-008, FR-016 (filters):** implemented and tested through the API; the TUI exposes sorting but has no profile, train-edit or filter screen.
- **FR-013:** connected journeys are offered when no direct train exists or when they are faster than the fastest direct train. Each leg is a separate ticket with its own reservation charge, so a connection is not offered for being cheaper.
- **FR-012 / FR-016:** open defect D-01: search lists a train in a class that has 0 seats on it; booking it fails safely (nothing booked or charged).
- **Data storage:** data is held in memory; file or database persistence (SRS 2.4, 3.x) is not implemented yet. The audit log is written to `data/audit.log`.
- **NFR-003:** validated with an in-process soak run; true uptime can only be measured on a deployment.

Appendix C: Requirement Traceability Matrix (completed)

Architecture and design references are to the SAD v1.0 (sections and figures). Code paths are under `RRS/04_IMPLEMENTATION_TESTING/source-code/src/`. Test Case IDs: UT/IT/ST from Test Plan §13.1; TC-nn are the cases added during the sprints (`test-cases.xlsx`); names in code font are GoogleTest cases. System Test IDs are the TUI system tests. Result as of 12.11.2026 on `main`: **Pass*** = passes, with open defect D-01 recorded against it.

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID	Result
1	FR-001	Register with unique email and a complex password	Auth & RBAC	§4.3 auth_register; §4.6 User	`auth/auth.cpp`, `auth/password.cpp`	TC-01 `RegisterValidPassengerSucceeds` `RegisterRejectsWeakPassword`	ST-TUI-01	Pass
2	FR-002	Reject a duplicate email	Auth & RBAC	§4.3 auth_register (ERR_DUPLICATE)	`auth/auth.cpp`	TC-02 `DuplicateEmailIsRejected` `DuplicateCheckIsCaseInsensitive`	-	Pass
3	FR-003	Authenticate and create a role-tagged session	Auth & RBAC	§4.3 auth_login; §4.6 Session	`auth/auth.cpp`, `auth/session.cpp`	TC-03 `ValidLoginReturnsPassengerSession` `AdminAccountGetsAdminRole`	ST-TUI-02	Pass

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID	Result
4	FR-004	Only admins can manage trains (RBAC)	Auth & RBAC; Facade	§3.9 RBAC (SEC-002)	`auth/rbac.cpp`, `inventory/inventory.c`	ST-01 `PassengerSessionIsForbiddenFromAdminFunction` `EditingTheRoleFieldDoesNotEscalatePrivileges`	ST-TUI-06	Pass
5	FR-005	View and update own profile	Auth & RBAC	§4.6 User	`auth/auth.cpp` `(auth_profile_get/update)`	TC-04 `ViewReturnsOwnDetails` `UpdateNameIsVisibleInTheProfile`	-	Pass
6	FR-006	End the session on idle timeout or logout	Auth & RBAC	§4.6 Session	`auth/session.cpp`, `auth/auth.cpp` `(auth_logout)`	TC-05 `SessionIsExpiredAtTheIdleTimeout` `LogoutEndsTheSession`	-	Pass
7	FR-007	Admin creates a train with halts and capacities	Inventory & Schedule	§4.3 train_create; §4.6 Train	`inventory/inventory.c`	TC-06 `AdminCreatesATrainAndCanReadItBack` `CreatedTrainBecomesSearchable`	ST-TUI-06	Pass
8	FR-008	Admin edits route, timings or capacities	Inventory & Schedule	§4.3 train_update	`inventory/inventory.c`	TC-07 `AdminEditsCapacitiesAndTheyAreStored` `EditedRouteReplacesTheOldOneInSearch`	-	Pass
9	FR-009	Admin retires a train; history stays viewable	Inventory & Schedule	§4.3 train_retire	`inventory/inventory.c`	TC-08 `RetiredTrainIsNoLongerBookableOrSearchable` `RetiredTrainRecordIsPreserved`	ST-TUI-08	Pass
10	FR-010	Halts must form a valid connected path	Inventory & Schedule; Search, Fare & Eco	§4.6 Route/Edge	`inventory/inventory.c`, `search/graph.cpp`	TC-09 `ConnectedRouteIsAccepted` `RepeatedStationIsRejected`	ST-TUI-06	Pass
11	FR-011	Capacity cannot drop below confirmed bookings	Inventory & Schedule	§4.4 ERR_CAPACITY_CONFLICT	`inventory/inventory.c`	TC-10 `ReductionBelowConfirmedBookingsIsRejected` `ReductionDownToConfirmedBookingsIsAllowed`	-	Pass
12	FR-012	Return all direct routes	Search, Fare & Eco	Fig. 4 Search; §4.3 search_routes	`search/search.cpp`, `search/graph.cpp`	UT-03, IT-01 `DirectSearchReturnsAllMatchingTrainsRanked` `TrainGoingTheOtherWayIsNotReturned`	ST-TUI-03, ST-TUI-09	Pass*
13	FR-013	Return connected (multi-leg) journeys	Search, Fare & Eco	Fig. 4 Search; §4.3 search_routes	`search/search.cpp`	IT-02 `MultiLegRouteViaBIsReturnedWhenNoDirectTrain` `FasterConnectionIsListedBeforeASlowDirectTrain`	-	Pass
14	FR-014	Fare by distance, class and concession	Search, Fare & Eco	§4.3 fare_calculate	`search/fare_eco.cpp`	UT-01, UT-02 `DirectSleeperNoConcessionMatchesHandComputedFare` `SeniorConcessionDiscountsTheDistanceCharge`	ST-TUI-03, ST-TUI-04	Pass

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID	Result
15	FR-015	Eco-score (CO2 per passenger)	Search, Fare & Eco	§4.3 eco_score	`search/fare_eco.cpp`	TC-11 `Co2AndScoreForADirectSleeperTrip` `AcClassesEmitMorePerPassenger`	ST-TUI-03	Pass
16	FR-016	Sort and filter by fare, time or eco-score	Search, Fare & Eco	Fig. 4 Search (search_rank added, see SAD v1.1)	`search/fare_eco.cpp`, `tui/main_tui.c`	TC-12 `SortByTimeFareAndEco` `Filters`	ST-TUI-03	Pass
17	FR-017	Search within the NFR-001 time limit	Search, Fare & Eco	Fig. 4 Search	`search/search.cpp`	ST-02, TC-13 `DefaultSearchMeetsNfr001` `ThreeTrainSearchMeetsNfr001`	-	Pass
18	FR-018	Booking and unique PNR only after payment succeeds	Booking & Waitlist; Mock payment	Fig. 2 Book Ticket; §4.3 booking_create	`booking/booking.c`, `facade/facade.cpp`	TC-14 `CreateBookingSuccessfulPaymentGeneratesConfirmedPNR` `MultipleBookingsGenerateDistinctPNRs`	ST-TUI-04, ST-TUI-07	Pass
19	FR-019	No booking when payment fails	Booking & Waitlist; Mock payment	Fig. 2 Book Ticket; Fig. 6 activity	`booking/booking.c`	IT-03 `PaymentFailureRejectsBookingAndCreatesNoPNR` `FailedPaymentBooksNothing`	-	Pass
20	FR-020	Waitlist when full, first come first served	Booking & Waitlist	Fig. 2 Book Ticket; §4.6 Waitlist	`booking/booking.c`, `booking/waitlist.c`	IT-04 `EnqueuesWaitlistWhenClassFullAndAutoPromotesOnCancel` `WaitlistEnqueuePreservesFcfsOrder`	ST-TUI-07	Pass
21	FR-021	Cancel a confirmed or waitlisted ticket	Booking & Waitlist	Fig. 3 Cancel; §4.3 booking_cancel	`booking/booking.c`	TC-15 `CancelTicketPermissionsAndIdempotency` `CancelWaitlistedTicketRemovesFromQueueAndShiftsPositions`	ST-TUI-05, ST-TUI-07	Pass
22	FR-022	Promote the waitlist head when a seat frees up	Booking & Waitlist	Fig. 3 Cancel; §4.3 waitlist_promote	`booking/booking.c`, `booking/waitlist.c`	IT-04 `EnqueuesWaitlistWhenClassFullAndAutoPromotesOnCancel` `CancelThenPromoteNeverExceedsCapacity`	ST-TUI-07	Pass
23	FR-023	Show booking and waitlist status	Booking & Waitlist; TUI	§4.5 UX (status visible)	`booking/booking.c`, `tui/main_tui.c`	TC-16 `PassengerBookingListRetrieval`	ST-TUI-05, ST-TUI-07	Pass
24	FR-024	No duplicate booking on retry	Booking & Waitlist	§4.3 idempotency key	`booking/booking.c`, `facade/facade.cpp`	ST-03 `IdempotentRetryReturnsExistingBookingWithoutDuplicate` `RetryAfterSuccessfulChargeReturnsSamePnrWithoutSecondCharge`	-	Pass
25	NFR-001	95% of searches within 2 s	Search, Fare & Eco	§3.9 DoS: bounded search time	`search/search.cpp`	ST-02 `DefaultSearchMeetsNfr001` `ThreeTrainSearchMeetsNfr001`	-	Pass
26	NFR-002	Booking confirmation within 1 s	Booking & Waitlist	Fig. 2 Book Ticket	`booking/booking.c`	TC-17 `NFR002BookingConfirmationTimeUnderNormalLoadMeets1sLimit`	-	Pass

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID	Result

27	NFR-003	99% availability	Facade; all modules	§4.4 error handling	`facade/facade.cpp`	TC-18
						`ValidRequestsSucceedAtLeast99PercentOverASoakRun`
						`ServiceKeepsRunningAfterBadInputAndFailedPayment`
							-	Pass

28	NFR-004	Search to booking in at most 6 screens	TUI	§4.5 UX (6 screens)	`tui/main_tui.c`	Code review	ST-TUI-03, ST-TUI-04	Pass

29	NFR-005	No duplicate booking on payment-timeout retry	Booking & Waitlist; Facade	§4.3 idempotency key	`booking/booking.c`, `facade/facade.cpp`	ST-03
						`TimeoutBeforeChargeLeavesNoBookingThenRetrySucceedsOnce`
						`RetryAfterSuccessfulChargeReturnsSamePnrWithoutSecondCharge`
							-	Pass

30	NFR-006	Modular code with tests per module	All (layered + facade)	§3.5 pattern; §4.1	`CMakeLists.txt`, `MODULES.md`	Code review	-	Pass
31	NFR-007	Engine testable without the UI	All	§4.1 testable without TUI	`include/*.h, tests/`	Code review	-	Pass

32	NFR-008	500 trains and 10,000 bookings	Search, Fare & Eco; Booking & Waitlist	§4.6; §4.9 adjacency list	`search/graph.cpp`, `inventory/inventory.c`, `booking/waitlist.c`	TC-19
						`FiveHundredTrainsAreStoredAndSearchable`
						`TenThousandBookingsAcrossFiveHundredTrains`
							-	Pass

33	NFR-009	No seat allocated twice, no data races	Booking & Waitlist	§4.1 centralised booking state	`booking/booking.c (mutex)`	ST-05
						`NFR009NoDoubleBookingUnder50ConcurrentRequestsForLastSeat`
						`NFR009SameSessionConcurrentBookingsNoDataRace`
							-	Pass

34	SEC-001	Salted password hashes only	Auth & RBAC	§3.9 Information disclosure	`auth/password.cpp`	ST-04
						`Sec001PasswordsAreStoredAsSaltedHashesOnly`
						`SamePasswordGetsDifferentSalts`
							-	Pass

35	SEC-002	Passenger cannot call admin functions	Facade / RBAC	§3.9 Elevation of privilege	`auth/rbac.cpp`, `inventory/inventory.c`	ST-01
						`Sec002PassengerCannotCallAdminFunctions`
						`Sec002PassengerCannotReadOrCancelAnotherPassengersBooking`
							-	Pass

36	SEC-003	Validate all user input	Facade / input validation	§3.9 Tampering	`all modules (validation on every call)`	TC-20
						`Sec003SearchRejectsMalformedInput`
						`Sec003BookingRejectsMalformedInput`
							-	Pass

37	SEC-004	Expired or invalid tokens rejected everywhere	Auth & RBAC	§3.9 Spoofing	`auth/session.cpp`	TC-21
						`Sec004ExpiredAndInvalidTokensAreRejectedEverywhere`
						`ExpiredTokenIsRejectedByEveryProtectedCall`
							-	Pass

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID	Result
38	SEC-005	No payment details stored	Booking & Waitlist; Mock payment	§3.9 Information disclosure	`booking/booking.c`	TC-22 `Sec005OnlyAPaymentFlagIsKept` `SEC005NoSensitivePaymentDataInBookingRecord`	-	Pass
39	SEC-006	Failed logins and admin actions audited	Audit logger	§3.9 Repudiation; §4.4	`audit/audit.cpp`, `auth/rbac.cpp`	TC-23 `Sec006FailedLoginsAndAdminActionsAreAudited` `ThreeFailedLoginsAndAnAdminActionAreAllLogged`	ST-TUI-02, ST-TUI-06	Pass

Full test lists, coverage and results: `docs/system_validation_report.md`, `docs/unit_test_report.md`, `RRS/04_IMPLEMENTATION_TESTING/test-cases.xlsx`.